RAPPORT DE PROJET

Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

Travail réalisé dans le cadre de l'UE

Systèmes Parallèles et distribués - OS202

Par les étudiants

ISIS KOMGUEM & MAEVA NOUKOUA

Soumis à l'appréciation de

M. JUVIGNY XAVIER

M. PARRET-FREAUD AUGUSTIN

Mars 2026

Palaiseau, France

Table des matières

Table des matières ii

Table des figures iii

Introduction i

1 Modélisation de Référence et Notions Clés ii
- 1.1 Description formelle du modèle ii
  - 1.1.1 Représentation de l'environnement ii
  - 1.1.2 Dynamique des phéromones ii
  - 1.1.3 Règle de déplacement ii
  - 1.1.4 Paramètres ii
- 1.2 Génération du terrain fractal ii
- 1.3 Architecture du code de référence iii

2 Environnement expérimental iv
- 2.1 Machine 1 — Isis (Intel Core i7-13650HX) iv
  - 2.1.1 Caractéristiques matérielles iv
  - 2.1.2 Visualisation (lstopo) iv
  - 2.1.3 Chaîne de compilation de la machine 1 v
- 2.2 Machine 2 — Maëva (MacBook Air) v
  - 2.2.1 Caractéristiques matérielles v
  - 2.2.2 Visualisation (lstopo) v
  - 2.2.3 Chaîne de compilation vi
- 2.3 Protocole pour assurer la reproductibilité vi
  - 2.3.1 Paramètres de simulation fixes vi
  - 2.3.2 Procédure de collecte vii
  - 2.3.3 Métriques de scalabilité vii
  - 2.3.4 Reproductibilité vii

3 Version de référence : analyse mono-cœur viii
- 3.1 Démarche viii
- 3.2 Résultats bruts viii
  - 3.2.1 Machine 1 — (i7-13650HX, Linux) viii
  - 3.2.2 Machine 2 —(MacBook Air, macOS) ix
- 3.3 Analyse ix
  - 3.3.1 Le rendu SDL masque le calcul ix
  - 3.3.2 Profil de calcul hors rendu ix
  - 3.3.3 Scalabilité en nombre de fourmis (m) x
  - 3.3.4 Comparaison entre machines x
  - 3.3.5 Indicateurs fonctionnels de l'ACO x
  - 3.3.6 Conclusion : identification du goulot d'étranglement xi

4 Vectorisation par passage SoA xii
- 4.1 Démarche xii
- 4.2 Résultats — version SoA (1 cœur) xii

4.2.1 Machine 1 — (i7-13650HX, Linux) . . . xii
4.3 Machine 2 — (MacBook Air, macOS) . . . xiii
4.4 Interprétation SoA vs AoS (focus machine 1) . . . xiii

5 Parallélisation OpenMP . . . xv
5.1 Démarche . . . xv
5.2 Résultats . . . xv
5.2.1 Machine 1 (i7-13650HX, Linux) . . . xv
5.2.2 Machine 2 (MacBook Air, macOS) . . . xv
5.3 Analyse . . . xv
5.4 Machine 1 : bonne accélération jusqu'à 16 threads . . . xvi
5.5 Machine 2 : absence totale de speedup OpenMP . . . xvii

6 Parallélisation distribuée : MPI . . . xix
6.1 Approche 1 : carte globale, fourmis réparties . . . xix
6.1.1 Principe . . . xix
6.1.2 Implémentation clé . . . xix
6.1.3 Résultats expérimentaux . . . xix
6.1.4 Analyse du goulot d'étranglement . . . xx
6.2 Approche 2 : décomposition de domaine . . . xx
6.2.1 Stratégie . . . xx
6.2.2 Gestion des bords (cellules fantômes) . . . xx
6.2.3 Gestion de la migration des fourmis . . . xxi
6.2.4 Stratégie pour mesurer l'accélération . . . xxi
6.3 Résultats expérimentaux — Machine 1 . . . xxi
6.4 Analyse des résultats (Machine 1) . . . xxii
6.5 Avantages et inconvénients . . . xxii

Références . . . xxiii

Table des figures

1 Architecture mémoire de la Machine 1 (i7-13650HX) — sortie `lstopo -no-io` iv
2 Topologie CPU de la Machine 2 (Maëva) obtenue avec `lstopo topology.svg` vi

Introduction

Les algorithmes d'*Intelligence en essaim* (*Swarm Intelligence*) s'inspirent du comportement collectif d'insectes sociaux. Des agents simples, dotés de capacités locales limitées, interagissent indirectement via l'environnement selon un mécanisme appelé **stigmergie**, permettant l'émergence d'un comportement global efficace sans coordination centralisée [2].

Parmi ces méthodes, l'**Ant Colony Optimisation** (ACO) [4] reproduit le comportement de fourragement des fourmis : en se déplaçant, elles déposent des *phéromones* sur leur trajet, influençant les choix des congénères. Les chemins les plus courts sont ainsi progressivement renforcés tandis que les traces inutilisées s'évaporent.

Dans ce projet, nous étudions une implémentation simplifiée d'ACO appliquée au problème de **fourragement sur un terrain fractal 2D**. L'objectif est de trouver un chemin de coût minimal entre un nid et une source de nourriture sur un terrain généré par un algorithme fractal.

Nous analyserons les performances de cette simulation selon plusieurs approches d'optimisation : instrumentation du code de référence, réorganisation des données, parallélisation en mémoire partagée avec **OpenMP** et parallélisation distribuée avec **MPI**. Les expériences sont réalisées sur deux machines afin d'évaluer l'impact de l'architecture matérielle sur les performances.

Lien Github du projet : `https://github.com/Napster7-0/OS202_project.git`

1 Modélisation de Référence et Notions Clés

1.1 Description formelle du modèle

1.1.1 Représentation de l'environnement

L'environnement est une grille cartésienne $N \times N$ ($N = 512$). Chaque cellule s stocke trois valeurs :

- $c(s) \in [0, 1]$: coût de traversée (généré par plasma fractal) ;
- $V_1(s) \in [0, 1]$: phéromone de type 1, guide vers la *nourriture* ;
- $V_2(s) \in [0, 1]$: phéromone de type 2, guide vers le *nid*.

1.1.2 Dynamique des phéromones

À chaque pas de temps, lorsqu'une fourmi visite la cellule s, les phéromones sont mis à jour selon :

$$V_i(s) \leftarrow \begin{cases} 1 & \text{si } s \text{ est la source cible du phéromone } i \\ \alpha \cdot \max_{s' \in \mathcal{N}(s)} V_i(s') + (1-\alpha) \cdot \frac{1}{4} \sum_{s' \in \mathcal{N}(s)} V_i(s') & \text{sinon} \end{cases} \tag{1}$$

où $\mathcal{N}(s)$ désigne les quatre voisins (haut, bas, gauche, droite) de s, et $\alpha \in [0, 1]$ est le **paramètre de bruit**.

Après chaque itération, tous les phéromones s'évaporent :

$$V_i(s) \leftarrow \beta \cdot V_i(s), \quad \forall s \tag{2}$$

où $\beta \in]0, 1[$ est le **coefficient d'évaporation**.

1.1.3 Règle de déplacement

À chaque sous-pas, une fourmi en position s choisit sa prochaine cellule selon :

- avec probabilité ε (**exploration**) : cellule voisine tirée uniformément parmi $\mathcal{N}(s) \setminus \{\text{obstacles}\}$;
- avec probabilité $1 - \varepsilon$ (**exploitation**) : cellule s^* maximisant $V_{i^*}(s')$, où $i^* = 1$ si la fourmi est non chargée, $i^* = 2$ si elle est chargée.

La fourmi avance tant que son budget de mouvement b (initialisé à 1) n'est pas épuisé : $b \leftarrow b - c(s_{\text{new}})$.

1.1.4 Paramètres

1.2 Génération du terrain fractal

Le terrain est généré par l'algorithme *midpoint displacement* récursif : une grille de $n \times n$ sous-grilles de taille $2^k + 1$ est initialisée aux coins, puis subdivisée récursivement en calculant les points médians avec une déviation bornée par $d \times \frac{ns}{2}$. Les valeurs sont ensuite normalisées dans $[0, 1]$.

TABLE 1 – Paramètres de la simulation de référence

Paramètre	**Symbole**	**Valeur**	**Rôle**
Nombre de fourmis	m	5 000	Agents simulés
Taille de la grille	N	512	$N \times N$ cellules
Exploration	ε	0.8	Proportion d'exploration
Bruit	α	0.7	Pondération max/moyenne
Évaporation	β	0.999	Décroissance des phéromones

1.3 Architecture du code de référence

Le code est organisé en quatre modules :

`fractal_land` Génération et stockage du terrain (tableau 1D de `double`).

`pheronome` Grille $(N+2)^2$ de paires $[V_1, V_2]$, avec cellules fantômes à -1 pour les bords. Méthodes : `mark_pheronome()`, `do_evaporation()`, `update()` (double-buffering).

`ant` Un objet par fourmi (**AoS**) : état, position (`SDL_Point`), graine aléatoire. Méthode `advance()` : boucle de sous-pas jusqu'à budget épuisé.

`ant_simu` Point d'entrée, boucle principale SDL, appel à `advance_time()` à chaque itération.

2 Environnement expérimental

Les expériences ont été conduites sur **deux machines distinctes** afin d'évaluer si les tendances de performance observées — profils de speedup, saturation, goulots d'étranglement — sont spécifiques à une architecture ou généralisables.

2.1 Machine 1 — Isis (Intel Core i7-13650HX)

2.1.1 Caractéristiques matérielles

Table 2 – Caractéristiques de la Machine 1

Composant	Valeur
Modèle processeur	Intel Core i7-13650HX (Raptor Lake, 13e gén.)
Architecture	Hybride P-cores / E-cores
Cœurs physiques	14 (6 P-cores + 8 E-cores)
Threads logiques	20 (P-cores : HT actif ; E-cores : 1 thread/cœur)
Fréquence max	4,9 GHz (P-core Turbo)
Cache L1d / L1i	48 Ko par P-core · 32 Ko par E-core
Cache L2	1 280 Ko par P-core · 2 048 Ko par groupe E-core
Cache L3 (partagé LLC)	24 Mo
RAM	16 Go DDR5
Système d'exploitation	Ubuntu 22.04 LTS (noyau Linux 6.x)

2.1.2 Visualisation (lstopo)

La figure 1 illustre la hiérarchie mémoire telle que rapportée par l'outil `hwloc/lstopo` [3]. On distingue clairement le nœud NUMA ;

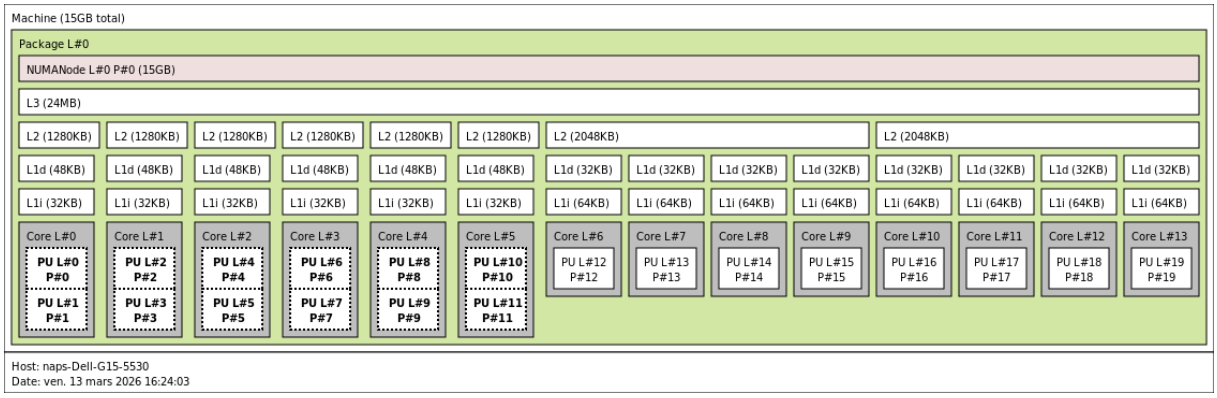

Figure 1 – Architecture mémoire de la Machine 1 (i7-13650HX) — sortie `lstopo -no-io`

L'architecture hybride introduit une **asymétrie de performance** entre threads. Les 12 threads des P-cores (6 cœurs × 2 HT) offrent la meilleure latence et la bande passante L2 la plus large. Les 8 threads des E-cores sont plus lents (fréquence et largeur de pipeline réduits) mais économes en énergie. Lors des benchmarks OpenMP, cette asymétrie se traduit par une saturation des gains au-delà de 12 threads, un plateau caractéristique de ce type de processeur hybride.

2.1.3 Chaîne de compilation de la machine 1

Table 3 – Chaîne d'outils et flags de compilation

Outil	Version (M1)	Flags principaux
`g++`	11.4.0	`-O2 -std=c++17 -fopenmp`
`mpic++`	OpenMPI 4.1	`-O2 -std=c++17 -fopenmp`
OpenMP	5.0	(runtime GCC)
OpenMPI	4.1	–
SDL2	2.0	(visualisation seulement)

2.2 Machine 2 — Maëva (MacBook Air)

2.2.1 Caractéristiques matérielles

Les caractéristiques matérielles de la machine ont été obtenues à l'aide des commandes `system_profiler SPHardwareDataType` et `sysctl`, qui constituent l'équivalent de `lscpu` sous macOS.

Table 4 – Caractéristiques de la Machine 2 (Maëva)

Composant	Valeur
Modèle processeur	Apple M4
Architecture	ARM64 (Apple Silicon)
Cœurs physiques	10 (4 Performance + 6 Efficiency)
Threads logiques	10
Cache L2 / L3	cache système partagé (non exposé via `sysctl`)
RAM	16 GB
Système d'exploitation	macOS

2.2.2 Visualisation (lstopo)

La topologie matérielle a été inspectée à l'aide de l'outil `hwloc` via la commande :

```
lstopo topology.svg
```

L'architecture du processeur Apple M4 est également **hybride**, avec une distinction entre cœurs haute performance et cœurs haute efficacité. Les **4 P-cores** disposent d'un cache L2 partagé de 16 MB et de caches L1 plus larges (128 KB données, 192 KB instructions), offrant la meilleure capacité de calcul. Les **6 E-cores** partagent un cache L2 de 4 MB et possèdent des caches L1 plus modestes (64 KB données, 128 KB instructions). Cette asymétrie introduit des différences de performance entre threads lors de l'exécution des versions parallèles OpenMP, les P-cores fournissant les meilleures performances tandis que les E-cores privilégient l'efficacité énergétique.

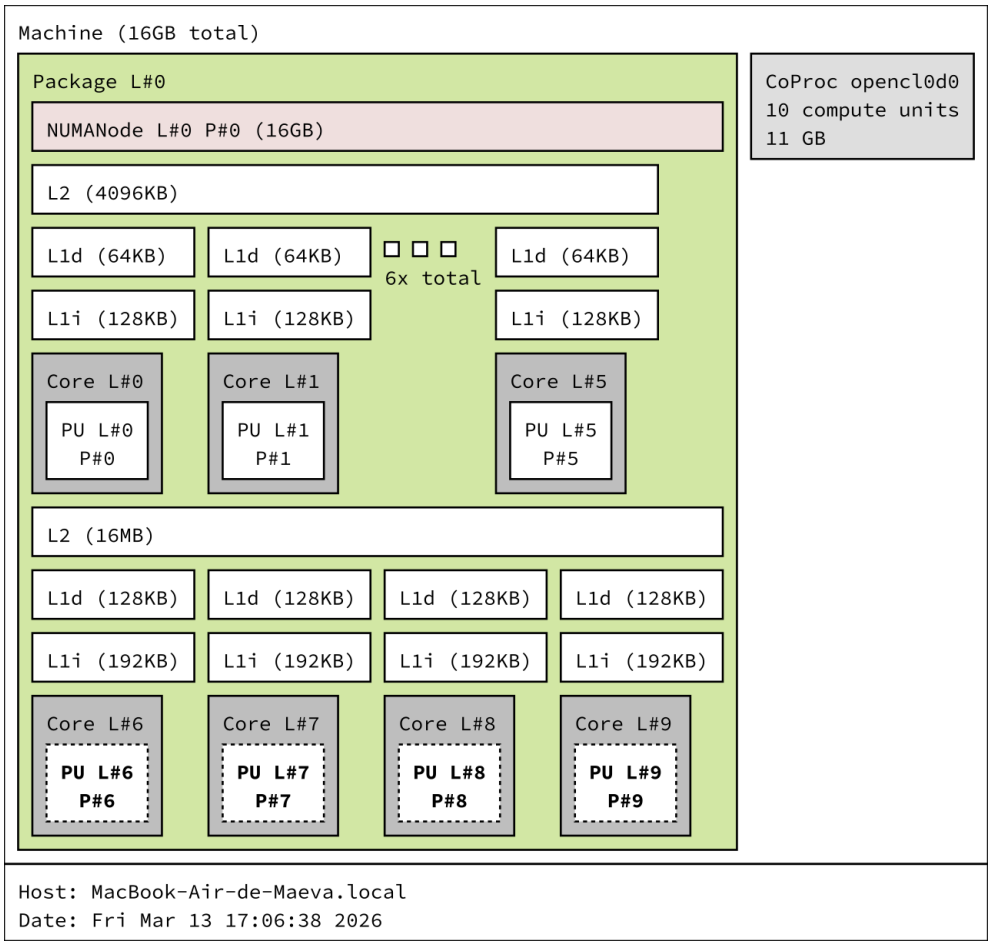

FIGURE 2 – Topologie CPU de la Machine 2 (Maëva) obtenue avec `lstopo topology.svg`

2.2.3 Chaîne de compilation

TABLE 5 – Chaîne d'outils et flags de compilation sur la machine Mac M4

Outil	Version	Flags principaux
`g++`	15.2.0 (Homebrew GCC)	`-O2 -std=c++17 -fopenmp`
`mpirun`	OpenMPI 5.0.9	exécution MPI
SDL2	2.32.10	visualisation uniquement

2.3 Protocole pour assurer la reproductibilité

2.3.1 Paramètres de simulation fixes

Pour assurer la comparabilité des mesures entre les deux machines, les paramètres de la simulation sont maintenus constants et communs(voir aussi le chapitre 1) :

- Grille $N = 512$ cellules par côté ;
- $m = 5\,000/10000$ fourmis ;
- Paramètres : $\varepsilon = 0.8$, $\alpha = 0.7$, $\beta = 0.999$;
- Graine aléatoire initiale : `2026` (reproductibilité).
- $nb_{threads}/nb_{proc} = 2/4/8/12/16/20$ en mode parrallèle.

2.3.2 Procédure de collecte

Chaque exécution instrumentée écrit ses données temporelles dans un fichier `profiling.csv` (une ligne par itération), selon le format :

```
iteration,t_ants_ms,t_evap_ms,t_update_ms,t_render_ms,food_count
0,4.1521,0.6489,0.0118,97.3214,0
1,4.2013,0.6601,0.0121,98.1042,0
...
```

Listing 1 – Format du fichier profiling.csv

Un programme d'analyse `profile_stats.exe` agrège ensuite ces données et calcule pour chaque phase : **moyenne**, **médiane**, **min**, **max** et **écart-type**. Les tableaux de résultats présentés dans les chapitres suivants sont issus de cette agrégation.

2.3.3 Métriques de scalabilité

Pour comparer les versions parallèles, on utilise les métriques classiques du calcul haute performance [7] :

$$S(p) = \frac{T_1}{T_p} \qquad \text{(accélération)} \tag{3}$$

$$E(p) = \frac{S(p)}{p} \qquad \text{(efficacité parallèle)} \tag{4}$$

où T_1 est le temps de la version séquentielle de référence et T_p le temps avec p unités de calcul. L'efficacité idéale est $E(p) = 1$ (100 %).

2.3.4 Reproductibilité

Reproductibilité fonctionnelle. Avec les mêmes paramètres et la même graine, les indicateurs métier (première nourriture ramenée au nid, quantité totale collectée) doivent rester stables entre runs sur la même machine.

Reproductibilité des performances. Les temps absolus (ms/it) ne sont *pas* transférables d'une machine à l'autre (fréquence CPU, hiérarchie mémoire, politique d'ordonnancement du système). En revanche, les **parts relatives par phase** et les **tendances de speedup** doivent être qualitativement similaires si les architectures sont comparables.

Version OpenMP. La race condition délibérée sur `mark_pheronome()` rend le comportement fonctionnel non strictement déterministe en mode multi-thread. Les indicateurs métier peuvent varier de ±5 % entre runs OpenMP, ce qui est acceptable pour une évaluation de scalabilité.

3 Version de référence : analyse mono-cœur

3.1 Démarche

La première étape consiste à *instrumenter* le code de référence (`00_src/`) sans modifier sa logique. La version instrumentée est placée dans `01_Profiling/` : chaque itération enregistre, dans un fichier `profiling.csv`, les durées des quatre phases de la boucle principale mesurées avec `std::chrono::steady_clock` :

— **ants** : avancement de toutes les fourmis (`advance_all()`) ;
— **evaporation** : application du coefficient β sur la grille (`do_evaporation()`) ;
— **update** : échange des buffers de phéromones (`update()`) ;
— **render** : dessin SDL du terrain et des fourmis.

Un programme séparé `profile_stats` agrège les données et calcule pour chaque phase la moyenne, le min, le max et la part relative (%). Les mesures ont été réalisées sur les deux machines (chapitre 2), pour $m = 5\,000$ et $m = 10\,000$ fourmis, sur des simulations de $\sim 15\,000$ itérations.

3.2 Résultats bruts

3.2.1 Machine 1 — (i7-13650HX, Linux)

Table 6 – Temps moyens par phase — Machine 1, $m = 5\,000$ fourmis

Phase	**Moy. (ms)**	**Min (ms)**	**Max (ms)**	**Part (%)**
Fourmis	4.43	1.89	8.50	5.62
Évaporation	0.64	0.23	1.74	0.81
Update	0.012	0.003	0.114	0.02
Rendu SDL	73.82	1.83	2029.3	93.56
Total	**78.90**	4.37	2032.2	100.0

Table 7 – Temps moyens par phase — Machine 1, $m = 10\,000$ fourmis

Phase	**Moy. (ms)**	**Min (ms)**	**Max (ms)**	**Part (%)**
Fourmis	6.45	2.92	14.54	16.99
Évaporation	0.47	0.24	1.44	1.23
Update	0.007	0.003	0.029	0.02
Rendu SDL	31.06	2.22	2026.7	81.77
Total	**37.98**	7.02	2035.6	100.0

3.2.2 Machine 2 —(MacBook Air, macOS)

TABLE 8 – Temps moyens par phase — Machine 2, $m = 5\,000$ fourmis

Phase	Moy. (ms)	Min (ms)	Max (ms)	Part (%)
Fourmis	1.24	0.83	10.91	9.49
Évaporation	0.15	0.10	0.71	1.12
Update	0.001	0.000	0.016	0.01
Rendu SDL	11.67	0.80	1030.2	89.38
Total	**13.06**	1.79	1031.7	100.0

TABLE 9 – Temps moyens par phase — Machine 2, $m = 10\,000$ fourmis

Phase	Moy. (ms)	Min (ms)	Max (ms)	Part (%)
Fourmis	2.58	2.06	10.67	11.91
Évaporation	0.15	0.11	0.51	0.70
Update	0.001	0.000	0.013	0.00
Rendu SDL	18.92	1.61	1033.3	87.38
Total	**21.65**	3.87	1035.8	100.0

3.3 Analyse

3.3.1 Le rendu SDL masque le calcul

Le rendu SDL représente entre **81 % et 94 %** du temps total selon la configuration. Cette domination s'explique par la synchronisation verticale (VSync) du pilote graphique : chaque appel à `SDL_RenderPresent()` se bloque jusqu'à la prochaine trame du moniteur (∼60 Hz, soit 16,7 ms). Les pics à plus de 2 000 ms correspondent à des gels momentanés du scheduler graphique.

Pour toute analyse algorithmique (vectorisation, OpenMP, MPI), le rendu doit être **exclu**. Les sections suivants portant sur la parallélisation s'appuient sur le mode sans rendu qui court-circuite SDL et enregistre directement dans le CSV.

3.3.2 Profil de calcul hors rendu

En écartant le rendu, on obtient le profil réel de la charge algorithmique (tableau 10). L'observation est cohérente sur les deux machines et les deux configurations.

TABLE 10 – Temps de calcul pur (hors rendu) et décomposition

Machine	m	Calcul (ms)	Fourmis (%)	Évap. (%)	Update (%)
Machine 1	5 000	5.08	87.2	12.5	0.2
Machine 1	10 000	6.93	93.2	6.7	0.1
Machine 2	5 000	1.39	89.4	10.6	0.1
Machine 2	10 000	2.73	94.4	5.6	0.0

La phase fourmis représente 87 à 94 % du calcul pur, l'évaporation 5 à 13 %, et l'update est négligeable ($< 0.2\,\%$). Par la loi d'Amdahl, les optimisations doivent se concentrer sur `advance_all()` pour produire un gain global significatif.

3.3.3 Scalabilité en nombre de fourmis (m)

Phase fourmis. En doublant m de 5 000 à 10 000, le temps de la phase fourmis augmente de :

— $\times 1.45$ sur la Machine 1 — scalabilité *sous-linéaire* ;
— $\times 2.08$ sur la Machine 2 — scalabilité *quasi-linéaire*.

La scalabilité sous-linéaire sur Machine 1 s'explique par les effets de cache : à $m = 5\,000$, la structure AoS (`vector<ant>`) tient partiellement dans le cache L2 (1 280 Ko par P-core), tandis qu'à $m = 10\,000$ le tableau déborde vers le L3 partagé (24 Mo), qui absorbe encore le tout. La pression sur le cache augmente donc moins que proportionnellement à m dans cette plage. Sur la Machine 2 (Apple Silicon), la quasi-linéarité indique un L2 unifié plus large ou une meilleure politique de prélecture matérielle (*hardware prefetcher*).

Phase évaporation. L'évaporation itère sur $N^2 = 512^2 = 262\,144$ cellules, *indépendamment de m*. Elle devrait être constante. C'est bien ce qu'on observe sur la Machine 2 (0.147 ms → 0.152 ms, stable à 3 %). Sur la Machine 1, une légère *baisse* (0.64 ms → 0.47 ms, −26 %) s'explique par un état du cache différent au moment du deuxième run (le cache L3 est déjà partiellement chaud avec les données de phéromones).

3.3.4 Comparaison entre machines

Table 11 – Rapport de performance mono-cœur Machine 1 vs Machine 2 ($m = 5\,000$)

Phase	Machine 1 (ms)	Machine 2 (ms)	Ratio M1/M2
Fourmis	4.43	1.24	$\times 3.6$
Évaporation	0.64	0.15	$\times 4.3$
Calcul total	5.08	1.39	$\times 3.7$

La Machine 2 est **3,6 à 4,3 fois plus rapide** sur chaque phase de calcul. Cet avantage reflète l'architecture Apple Silicon (M-series) : pipeline plus large, latence mémoire réduite grâce au SoC unifié (processeur et DRAM sur le même die), et gestion efficace du cache. Ce rapport de performance mono-cœur ne préjuge pas des résultats en mode parallèle, où la hiérarchie mémoire joue un rôle différent.

3.3.5 Indicateurs fonctionnels de l'ACO

Les mesures permettent également d'évaluer la qualité de la solution produite, indépendamment des temps d'exécution.

Table 12 – Indicateurs fonctionnels de l'algorithme ACO

Machine	m	1re nourr. (itér.)	Nourr. totale	Taux (nourr./itér.)
Machine 1	5 000	1 747	54 123	3.6
Machine 1	10 000	1 517	137 090	9.1
Machine 2	5 000	2 942	81 946	5.2
Machine 2	10 000	1 720	197 347	12.7

Convergence. La différence d'itération de première convergence entre les machines (1 747 vs 2 942 pour $m = 5\,000$) est d'origine *stochastique* : les graines aléatoires initiales et le terrain fractal diffèrent entre les deux environnements. Ce n'est pas un indicateur de performance matérielle.

Effet sur-linéaire de m. Doubler m produit un gain *super-linéaire* en nourriture collectée : $\times 2.5$ sur M1, $\times 2.4$ sur M2. Ce comportement est cohérent avec la convergence sur-linéaire mentionnée dans le sujet de TP [5] : davantage de fourmis renforcent plus vite les phéromones du chemin optimal, ce qui accélère exponentiellement la convergence de toute la colonie.

3.3.6 Conclusion : identification du goulot d'étranglement

La phase fourmis (`advance_all()`) est le seul goulot d'étranglement algorithmique, représentant 87 à 94 % du calcul pur sur les deux machines et les deux configurations testées. L'évaporation ($\sim 10\,\%$) constitue un cible secondaire dont le gain est limité par la bande passante mémoire. Les optimisations suivantes cibleront ces deux phases par ordre d'impact :

1. Refactoring AoS $\rightarrow$ SoA (`02_StructureOfArray/`) pour améliorer la localité mémoire et permettre la vectorisation SIMD ;
2. Parallélisation OpenMP de la boucle fourmis et de l'évaporation (`03_Parallele_01/`, chapitre 5) ;
3. Distribution des fourmis entre processus MPI (`04_Parallele_02/`, chapitre 6).

4 Vectorisation par passage SoA

4.1 Démarche

Le refactoring AoS → SoA consiste à remplacer le tableau d'objets `vector<ant>` par trois tableaux distincts, un pour chaque attribut :

```
class ant_colony {
    std::vector<state_t>    m_states;     // etat de chaque fourmi
    std::vector<position_t> m_positions; // position (x, y)
    std::vector<std::size_t> m_seeds;   // graine aleatoire
public:
    void advance_all(pheronome& phen, const fractal_land& land,
                     const position_t& pos_food,
                     const position_t& pos_nest,
                     std::size_t& cpteur_food);
};
```

Listing 2 – Structure SoA dans ant_colony (ant.hpp)

Cette organisation garantit que l'accès à un seul champ (par exemple les graines de tous les agents) est *contigu* en mémoire, ce qui favorise la prélecture matérielle et la vectorisation par le compilateur.

4.2 Résultats — version SoA (1 cœur)

Cette section présente les mesures de la version `02_StructureOfArray/` exécutée en mono-cœur (sans parallélisation OpenMP active), dans le même esprit que le chapitre 3. Les statistiques sont extraites de `profiling.csv` via `profile_stats`.

4.2.1 Machine 1 — (i7-13650HX, Linux)

Table 13 – SoA mono-cœur — Machine 1, $m = 5\,000$ fourmis

Phase	Moy. (ms)	Min (ms)	Max (ms)	Part (%)
Fourmis	3.9887	1.8321	8.9144	8.42
Évaporation	0.6109	0.2644	1.8508	1.29
Update	0.0115	0.0028	0.0394	0.02
Rendu SDL	42.7452	1.9377	2014.2910	90.26
Total	**47.3562**	4.4607	2021.7478	100.00

Table 14 – SoA mono-cœur — Machine 1, $m = 10\,000$ fourmis

Phase	Moy. (ms)	Min (ms)	Max (ms)	Part (%)
Fourmis	6.8784	3.6581	15.9118	5.24
Évaporation	0.4994	0.2271	1.2067	0.38
Update	0.0086	0.0028	0.0328	0.01
Rendu SDL	123.7647	2.0283	2043.6808	94.37
Total	**131.1510**	8.1222	2051.1610	100.00

Indicateurs fonctionnels observés :

— $m = 5\,000$: première nourriture au nid à l'itération 2585, nourriture finale 30324 ;
— $m = 10\,000$: première nourriture au nid à l'itération 1604, nourriture finale 141245.

4.3 Machine 2 — (MacBook Air, macOS)

Table 15 – SoA mono-cœur — Machine 2, $m = 5\,000$ fourmis

Phase	**Moy. (ms)**	**Min (ms)**	**Max (ms)**	**Part (%)**
Fourmis	1.2668	1.0220	7.9970	8.48
Évaporation	0.1521	0.1050	0.6720	1.02
Update	0.0011	0.0000	0.0270	0.01
Rendu SDL	13.5226	1.0760	1031.9500	90.50
Total	**14.9426**	2.2930	1033.2320	100.00

Table 16 – SoA mono-cœur — Machine 2, $m = 10\,000$ fourmis

Phase	**Moy. (ms)**	**Min (ms)**	**Max (ms)**	**Part (%)**
Fourmis	2.2499	2.0440	7.6410	12.08
Évaporation	0.1359	0.1050	0.5400	0.73
Update	0.0009	0.0000	0.0060	0.00
Rendu SDL	16.2449	1.3680	1054.4740	87.19
Total	**18.6315**	3.6790	1056.9160	100.00

Indicateurs fonctionnels observés :
— $m = 5\,000$: première nourriture au nid à l'itération 2585, nourriture finale 30827 ;
— $m = 10\,000$: première nourriture au nid à l'itération 1604, nourriture finale 155383.

4.4 Interprétation SoA vs AoS (focus machine 1)

Comme au chapitre 3, le rendu SDL domine (90–94%) et masque le coût algorithmique. La comparaison SoA vs AoS doit donc se faire sur le **calcul pur**, défini par :

$$T_{\text{calcul}} = T_{\text{fourmis}} + T_{\text{évaporation}} + T_{\text{update}}$$

Table 17 – Comparaison AoS vs SoA sur Machine 1 (hors rendu)

Config.	**AoS (ms)**	**SoA (ms)**	**Speedup SoA**	**Variation**
$m = 5\,000$	5.08	4.61	1.10×	−9.3%
$m = 10\,000$	6.93	7.39	0.94×	+6.6%

L'effet SoA est donc **mitigé** sur la machine 1 :
— pour $m = 5\,000$, SoA apporte un gain net d'environ 10% en calcul pur, cohérent avec une meilleure localité mémoire sur les champs les plus sollicités ;
— pour $m = 10\,000$, SoA devient légèrement plus lent qu'AoS, ce qui suggère que le bénéfice de localité est compensé par d'autres coûts (accès plus dispersés, pression cache, variabilité stochastique du trajet des fourmis).

Sur les indicateurs fonctionnels, les écarts AoS/SoA (itération de première nourriture, nourriture finale) restent difficiles à attribuer uniquement à la structure mémoire, car

l'algorithme est stochastique (graines, trajectoires et renforcement des phéromones). La conclusion robuste est donc : **SoA n'apporte pas un gain systématique en mono-cœur sur ce code tel quel**, mais prépare une base plus favorable pour la vectorisation SIMD et la parallélisation OpenMP présentées dans la suite du chapitre.

5 Parallélisation OpenMP

5.1 Démarche

La parallélisation est implémentée dans `03_Parallele_01/` à l'aide de deux directives `#pragma omp parallel for` : une sur la boucle fourmis (`advance_all()`) avec `schedule(dynamic, 16)` pour tenir compte de la variabilité des sous-pas par fourmi, et une sur la boucle d'évaporation avec `schedule(static)` pour sa charge parfaitement régulière.

Les expériences font varier `OMP_NUM_THREADS` de 1 à 20 (Machine 1, 20 threads logiques disponibles) et de 1 à 20 (Machine 2). Le mode `-no-render` est activé sur les deux machines afin d'isoler le coût purement calculatoire (la colonne `render_ms` vaut 0 dans tous les tableaux). Chaque mesure porte sur 15 000 itérations, graine fixée à 2585 pour assurer la comparabilité.

5.2 Résultats

5.2.1 Machine 1 (i7-13650HX, Linux)

Table 18 – Temps par phase selon le nombre de threads — Machine 1 (ms/it)

Threads p	**Fourmis**	**Évaporation**	**Update**	**Total** T_p
1	2.3341	0.2408	0.0037	2.578
2	1.2257	0.1171	0.0039	1.347
4	0.6318	0.0625	0.0049	0.696
8	0.3755	0.0543	0.0060	0.436
12	0.2800	0.0525	0.0063	0.338
16	0.2390	0.0501	0.0061	0.295
20†	0.3687	0.1973†	0.0056	0.572†

Table 19 – Accélération $S(p)$ et efficacité $E(p)$ — Machine 1

Threads p	T_p **(ms/it)**	$S(p)$	$E(p)$ **(%)**
1	2.578	1.000	100.0
2	1.347	1.915	95.7
4	0.696	3.705	92.6
8	0.436	5.919	74.0
12	0.338	7.620	63.5
16	0.295	8.729	54.6
20†	0.572	4.512	22.6†

† Régression sévère par rapport à 16 threads ; voir l'analyse §5.4.

5.2.2 Machine 2 (MacBook Air, macOS)

5.3 Analyse

Table 20 – Temps par phase selon le nombre de threads — Machine 2 (ms/it)

Threads p	**Fourmis**	**Évaporation**	**Update**	**Total** T_p
1	1.0141	0.1072	0.0006	1.122
2	1.0126	0.1068	0.0006	1.120
4	1.0090	0.1064	0.0006	1.116
8	1.0201	0.1087	0.0006	1.129
12	1.0195	0.1088	0.0007	1.129
16	1.0128	0.1069	0.0006	1.120
20	1.0512	0.1153	0.0007	1.167

Table 21 – Accélération $S(p)$ et efficacité $E(p)$ — Machine 2

Threads p	T_p **(ms/it)**	$S(p)$	$E(p)$ **(%)**
1	1.122	1.000	100.0
2	1.120	1.002	50.1
4	1.116	1.006	25.1
8	1.129	0.994	12.4
12	1.129	0.994	8.3
16	1.120	1.002	6.3
20	1.167	0.961	4.8

5.4 Machine 1 : bonne accélération jusqu'à 16 threads

Régime quasi-idéal (1 à 4 threads). L'efficacité est remarquablement haute en basse parallélisation : $E(2) = 95.7\,\%$, $E(4) = 92.6\,\%$. Cela confirme que la phase fourmis est *embarrassingly parallel* : les 5 000 fourmis s'exécutent sans contention notable sur les écritures de phéromones à faible nombre de threads.

Loi d'Amdahl. En prenant la meilleure mesure (16 threads, $S(16) = 8.73$), on extrait la fraction séquentielle résiduelle s par la relation d'Amdahl :

$$S(p) = \frac{1}{s + \dfrac{1-s}{p}} \quad \Longrightarrow \quad s = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}$$

$$s \approx \frac{\frac{1}{8.73} - \frac{1}{16}}{1 - \frac{1}{16}} \approx 5.56\,\% \qquad \Rightarrow \qquad S_\infty \approx \frac{1}{0.056} \approx 17.9$$

Environ 5.6 % du temps est séquentiel (réduction du compteur de nourriture, swap de buffers, initialisation des régions parallèles). La partie parallélisable atteint donc $f \approx 94.4\,\%$.

Saturation entre 8 et 16 threads. L'efficacité s'érode progressivement : E passe de 74.0 % à 8 threads à 54.6 % à 16 threads. Deux facteurs se combinent :

— **Architecture hybride.** Les 12 premiers threads logiques utilisent les 6 P-cores (via l'Hyper-Threading, 2 threads/cœur), qui offrent la meilleure latence. Les threads 13 à 20 mobilisent les 8 E-cores, à fréquence et bande passante réduites, introduisant un déséquilibre croissant de charge.

— **Évaporation memory-bound.** Le temps d'évaporation atteint un plateau dès 8 threads (0.054 → 0.050 ms/it de 8 à 16 threads), signe que la bande passante mémoire pour les ≈ 4 Mo du buffer de phéromones est saturée. Cette phase ne bénéficie plus du parallélisme au-delà de 8 threads.

Régression sévère à 20 threads. Le passage de 16 à 20 threads provoque une **régression majeure** : $T_{20} = 0.572$ ms contre $T_{16} = 0.295$ ms, soit un facteur $\times 1.94$ de dégradation. Le speedup tombe de 8.73 à 4.51. L'explication est principalement portée par l'évaporation :

$$T_{\text{évap}}(16) = 0.050 \text{ ms} \quad \longrightarrow \quad T_{\text{évap}}(20) = 0.197 \text{ ms} \quad (+\,294\,\%)$$

À 20 threads, le scheduler OpenMP distribue le travail sur la totalité des 20 threads logiques, incluant tous les E-cores. Les E-cores partagent leur L2 (2 Mo pour 4 cœurs), et leur accès simultané à la grille de phéromones (≈ 4 Mo) entraîne des **défauts de cache en rafale** et une contention sévère sur le bus mémoire. Parallèlement, à 20 threads, le système d'exploitation ne dispose plus de cœurs libres pour gérer les interruptions et le scheduler, ajoutant une latence de *context switch* non-préemptible.

La configuration **optimale sur Machine 1 est 16 threads** ($S = 8.73$, $E = 54.6\,\%$).

5.5 Machine 2 : absence totale de speedup OpenMP

Observation centrale. Sur la Machine 2, le temps de calcul reste **quasi-constant** quel que soit le nombre de threads : $T_p \in [1.116,\ 1.167]$ ms/it pour tout $p \in [1, 20]$. L'accélération ne dépasse jamais $S(p) = 1.006$ ($p = 4$), et l'efficacité suit la loi $E(p) \approx 100/p\,\%$, ce qui caractérise très précisément un système dont le **speedup réel vaut exactement 1** en toutes circonstances :

$$E(p) = \frac{S(p)}{p} \approx \frac{1}{p} \quad \Longrightarrow \quad S(p) \approx 1 \quad \forall\, p$$

Cause : surcoût OpenMP ≥ gain de calcul. À 1 thread, la phase fourmis prend ≈ 1.01 ms pour 5 000 fourmis, soit ≈ 202 ns par fourmi. Avec `schedule(dynamic, 16)`, on génère $\lceil 5000/16 \rceil = 313$ tâches à distribuer entre les threads. Le coût de gestion de chaque tâche (dispatch, accès à la file de travaux, barrière finale) est de l'ordre de la centaine de nanosecondes sur le runtime OpenMP de macOS (`libomp` d'LLVM via Apple Clang). Le ratio *surcoût/calcul utile par tâche* est donc proche de 1, ce qui annule complètement le bénéfice du parallélisme.

Architecture mémoire unifiée (Apple Silicon). Sur les processeurs Apple Silicon (M-series), l'architecture *Unified Memory* place processeur et RAM sur le même die avec une bande passante très élevée *par cœur*. La phase fourmis dans la version SoA accède à plusieurs tableaux contigus (coordonnées, états, graines), ce qui favorise la prélecture matérielle (*prefetching*) sur *un seul cœur*. En ajoutant des threads, tous les cœurs rivalisent pour la même bande passante mémoire unifiée : le débit total stagne sans diminuer le temps de calcul par thread, qui reste limité par les latences d'accès à la carte de phéromones partagée en lecture.

Bilan comparatif. La Machine 2 est intrinsèquement dans un régime où la granularité de la tâche est trop fine pour qu'OpenMP soit utile à $m = 5\,000$ fourmis. Un gain serait observable soit avec $m \gg 5\,000$ (granularité accrue), soit avec un `schedule(static)` strict qui élimine le surcoût de dispatch dynamique, au prix d'un équilibrage moins bon sur terrain fractal non uniforme.

Table 22 – Résumé comparatif des deux machines — OpenMP

Machine	T_1 **(ms)**	$S_{\mathbf{max}}$	$p_{\mathbf{opt}}$	$E(p_{\mathbf{opt}})$ **(%)**
Machine 1 (i7-13650HX)	2.579	8.73	16	54.6
Machine 2 (Apple M-series)	1.122	1.01	4	25.1

Bien que la Machine 2 soit plus rapide en mono-cœur (1.12 vs 2.58 ms), c'est la Machine 1 qui tire le meilleur parti d'OpenMP grâce à son architecture multi-cœurs homogène (en dessous de 16 threads) et à la plus grande granularité relative de la tâche par rapport au surcoût de threading.

6 Parallélisation distribuée : MPI

6.1 Approche 1 : carte globale, fourmis réparties

6.1.1 Principe

Dans cette première approche, chaque processus MPI possède une **copie complète** de la carte de phéromones ($N^2 \approx 262\,000$ cellules). Les $m = 5\,000$ fourmis sont réparties équitablement : le processus de rang r gère les fourmis $[r \cdot \lfloor m/P \rfloor,\ (r+1) \cdot \lfloor m/P \rfloor)$.

Le cycle d'une itération comprend quatre étapes :

1. **Avancement local** : chaque processus fait avancer ses m/P fourmis et met à jour son buffer de phéromones local via `mark_pheronome()`.
2. **Évaporation partielle** : chaque processus applique l'évaporation sur sa plage de lignes $[i_{\text{start}}, i_{\text{end}})$ uniquement (via `do_evaporation_range()`), répartissant ainsi la charge.
3. **Synchronisation globale** : `MPI_Allreduce(MPI_IN_PLACE, buf, MPI_MAX)` fusionne les buffers en choisissant la valeur maximale pour chaque cellule (conformément à la spécification du projet : lorsque deux processus ont mis à jour la même cellule, on retient la plus grande valeur).
4. **Update** : swap buffer → carte principale sur tous les processus.

6.1.2 Implémentation clé

La méthode `allreduce_buffer_max()` dans `pheronome.hpp` est le cœur de la synchronisation :

```
void allreduce_buffer_max() {
    double* buf = reinterpret_cast<double*>(m_buffer_pheronome.data
   ());
    std::size_t count = m_buffer_pheronome.size() * 2;
    MPI_Allreduce(MPI_IN_PLACE, buf,
                  static_cast<int>(count),
                  MPI_DOUBLE, MPI_MAX,
                  MPI_COMM_WORLD);
}
```

Listing 3 – Synchronisation MPI des phéromones

Le volume échangé à chaque itération est :

$$V = (N+2)^2 \times 2 \times 8 \text{ octets} = 514^2 \times 16 \approx 4{,}2 \text{ Mo}$$

6.1.3 Résultats expérimentaux

Table 23 – Temps par phase — version MPI Approche 1 (ms/it)

Processus P	Fourmis	Évaporation	Comm MPI	Total
1	1.58	0.16	0.00	1.57 [1]
2	0.80	0.08	0.65	1.51

1. Version MPI sans SDL, légèrement plus rapide que la version AoS SDL.

Table 24 – Accélération et efficacité — MPI Approche 1

Processus P	T_P **(ms/it)**	$S(P)$	$E(P)$ **(%)**
1	1.57	1.0	100
2	1.51	1.0	52

6.1.4 Analyse du goulot d'étranglement

L'accélération à 2 processus est quasi nulle : le gain de calcul (0.8 ms sur les fourmis + 0.08 ms sur l'évaporation = ≈ 0.86 ms économisés) est entièrement annulé par le coût de `MPI_Allreduce` (0.65 ms). L'efficacité tombe à 52% dès 2 processus.

La communication représente $\frac{0.65}{1.57} \approx 41\%$ du temps total à $P = 2$. La loi d'Amdahl appliquée à ce cas prédit une accélération maximale très limitée :

$$S_{\max} = \frac{1}{f_{\text{comm}}} = \frac{1}{0.41} \approx 2.4$$

L'Approche 1 n'est rentable que si le rapport m/N^2 est très élevé (beaucoup de fourmis sur une grille petite), ce qui n'est pas le cas ici.

6.2 Approche 2 : décomposition de domaine

6.2.1 Stratégie

Dans cette seconde approche, chaque processus ne gère qu'une **bande horizontale** de la carte de phéromones. Le processus r est responsable des lignes $[y_{\min}(r),\ y_{\max}(r)]$ et ne conserve en mémoire que cette zone (plus deux lignes fantômes). Les fourmis sont stockées localement en SoA, et **migrent** vers le voisin du haut ou du bas lorsqu'elles franchissent une frontière.

La stratégie retenue reste dans le cadre MPI "classique" du cours : `MPI_Scatterv`, `MPI_Alltoallv`, `MPI_Gatherv`, `MPI_Sendrecv`.

Initialisation (bucket spatial + Scatterv). Le rang 0 génère les m fourmis puis les répartit par "bucket" selon leur coordonnée y (appartenance à une bande). On calcule :

— `sendcounts[r]` : nombre de fourmis initiales pour le rang r ;

— `displs[r]` : déplacement dans les tableaux aplatis.

Les tableaux SoA aplatis (positions, états, graines) sont ensuite distribués avec `MPI_Scatterv`. Ce point est important : on évite un broadcast de toutes les fourmis à tous les processus.

Boucle d'itération locale. À chaque itération, le rang r :

1. fait avancer ses fourmis locales ;
2. applique l'évaporation sur sa bande locale ;
3. échange les lignes fantômes avec $r - 1$ et $r + 1$;
4. exporte les fourmis sorties de la bande (migrations).

6.2.2 Gestion des bords (cellules fantômes)

Pour calculer `mark_pheronome()` sur les lignes frontières, chaque processus a besoin des valeurs de phéromones de la ligne immédiatement au-dessus (processus $r - 1$) et en dessous (processus $r+1$). On utilise deux lignes fantômes mises à jour avec `MPI_Sendrecv` :

— envoi de la première ligne réelle vers $r-1$ et réception de la ghost basse ;
— envoi de la dernière ligne réelle vers $r+1$ et réception de la ghost haute.

Volume théorique minimal échangé par itération pour les phéromones :

$$V_{\text{bords}} = 2 \times N \times 2 \times 8 = 2 \times 512 \times 16 \approx 16\,\text{Ko}$$

soit environ 260× moins que l'Approche 1 (`Allreduce` global).

6.2.3 Gestion de la migration des fourmis

Une migration point-à-point naïve (plusieurs `MPI_Send` par fourmi) fonctionne, mais scale mal. Pour pallier à ça, on va :

1. remplir des buckets `out_up` et `out_down` ;
2. échanger les tailles avec les voisins (`MPI_Sendrecv`) ;
3. échanger les buffers compacts en une seule fois par voisin ;
4. intégrer les fourmis reçues dans les tableaux SoA locaux.

Si l'on veut généraliser au cas 2D (découpage en tuiles), la même logique devient un **bucket sort global** par propriétaire avec `MPI_Alltoallv` : chaque rang envoie un paquet compact à chaque rang destination.

Cette migration est potentiellement déséquilibrée (rang contenant la fourmilière plus chargé), mais avec m grand et ε élevé, la distribution tend à se lisser.

6.2.4 Stratégie pour mesurer l'accélération

On mesure en mode `-no-render` :
— T_P : temps moyen par itération (médiane de 3 runs), pour $P = 1, 2, 4, 8, \ldots$;
— $S(P) = T_1/T_P$ (accélération) ;
— $E(P) = S(P)/P$ (efficacité) ;
— fraction communication : $f_{\text{comm}} = t_{\text{comm}}/(t_{\text{ants}} + t_{\text{evap}} + t_{\text{comm}})$.

Attendu : la courbe $S(P)$ de l'Approche 2 dépasse nettement l'Approche 1 dès $P \geq 2$, car la communication passe d'un `Allreduce` global à de petits échanges de bords + migrations compactées.

Mise en œuvre réalisée. dans `04_Parallele_02/ant_simu_mpi_domain.cpp` avec :
— décomposition de domaine 1D (bandes horizontales),
— distribution initiale des fourmis par buckets + `MPI_Scatterv`,
— échange de ghost rows par `MPI_Sendrecv`,
— migration des fourmis par paquets via `MPI_Alltoallv`,
— profilage par itération écrit dans `profiling_mpi_domain.csv`.

Le Makefile inclut deux cibles pour l'évaluation :
— `make run_domain_profile` : exécute des runs $P = 1, 2, 4, 8$;
— `make summary_domain_profile` : calcule `mean_total_ms`, `speedup`, `efficiency_pct`.

6.3 Résultats expérimentaux — Machine 1

Indicateurs fonctionnels observés sur le run $P = 8$: première nourriture au nid à l'itération 2175.

Table 25 – Approche 2 (décomposition de domaine) — Machine 1

P	**Total**	$S(P)$	$E(P)$	**Fourmis**	**Évap.**	**Comm bord**	**Comm mig.**	**Update**
1	2.7828	1.000	100.0%	2.2958	0.3110	0.0011	0.0005	0.0032
2	1.6942	1.643	82.1%	1.4138	0.1824	0.0105	0.2298	0.0016
4	0.9856	2.823	70.6%	0.8713	0.0509	0.0099	0.2824	0.0007
8	0.6658	4.180	52.2%	0.5782	0.0309	0.0159	0.3045	0.0006

6.4 Analyse des résultats (Machine 1)

L'approche 2 accélère correctement jusqu'à 8 processus :

$$S(8) = 4.18, \qquad E(8) = 52.2\%.$$

Le passage de $P = 1$ à $P = 8$ réduit le temps moyen d'itération de 2.78 ms à 0.67 ms.

La phase fourmis diminue presque linéairement avec P (2.30 $\rightarrow$ 0.58 ms), ce qui confirme que la charge de calcul locale est bien distribuée. L'évaporation baisse aussi nettement (0.31 $\rightarrow$ 0.03 ms), conformément à la décomposition de la carte.

Le facteur limitant devient la communication de migration : `comm_migrate_ms` augmente avec P (0.0005 $\rightarrow$ 0.3045 ms) et domine la communication totale à partir de $P = 4$. Ce comportement est cohérent avec la méthode : plus le nombre de bandes augmente, plus les franchissements inter-domaines sont fréquents.

La communication de bord reste faible (0.01–0.016 ms), ce qui valide le bénéfice majeur de l'approche 2 par rapport à l'approche 1 : on remplace un `Allreduce` global coûteux par des échanges locaux de voisinage.

6.5 Avantages et inconvénients

Table 26 – Comparaison Approche 1 vs Approche 2

Critère	**Approche 1**	**Approche 2**
Mémoire par processus	$O(N^2)$ (carte entière)	$O(N^2/P)$ (bande)
Communication/itération	$\approx$ 4.2 Mo	$\approx$ 16 Ko + migration compactée
Équilibrage	Parfait (fourmis)	Variable (migration)
Complexité d'impl.	Faible	Élevée
Efficacité attendue	Faible ($P > 2$)	Bonne (P grand)

Références

[1] Anthropic. Claude ai, 2024.

[2] Gerardo Beni and Jing Wang. Swarm intelligence in cellular robotic systems. *Proceedings of the NATO Advanced Workshop on Robots and Biological Systems*, 1989.

[3] François Broquedis, Jérôme Clet-Ortega, Stéphanie Moreaud, Nathalie Furmento, Brice Goglin, Guillaume Mercier, Samuel Thibault, and Raymond Namyst. hwloc : A generic framework for managing hardware affinities in HPC applications. In *Proceedings of the 18th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pages 180–186, 2010.

[4] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. Ant system : Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1) :29–41, 1996.

[5] ENSTA Paris. Sujet de projet — OS202 : Optimisation par colonies de fourmis sur un paysage fractal, 2026. Document interne, cours OS202 Systèmes Distribués.

[6] GitHub. Github copilot, 2024.

[7] William Gropp, Ewing Lusk, and Anthony Skjellum. *Using MPI : Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 2nd edition, 1999.